```
       if (this.hashTable.contains(key))
{
        PriorityQueueNode node = this.hashTable.get(key);
         node.setValue(value);
        return node.updatePriority(node.getCounter());
      } else if (this.getSize() >= this.maxSize) {
        this.evictOneEntry();
                                          #3
      }
      PriorityQueueNode newNode = this.elements.add(key,
value, 1);
      this.hashTable.put(key, newNode);
      return true;
    } finally
{
#4

  writeLock.unlock();
           #5
    }
}
```

In these examples, we are still using the hash table to store references to the priority queue nodes, like we were storing references to the linked list nodes for LRU. While for the latter this makes sense to keep the constant running time for all operations, as explained in section 7.4.2, for LFU cache we can as well store just values in the hash table; this makes implementation simpler, as shown on our repo, without changing the running time for the operation.

Take a minute to think about it, before reading the explanation. The reason, as you should have figured out, is simple: both in `set` and `get` we call `updatePriority` on the priority queue; if we have a reference to the node to update,